COSC 330 – Final Project Demo Guide

Student: Nicholas Armenta **Course:** COSC 330 – Systems Programming **Due Date:** 11/05/2026

Overview

This guide walks through how to run every part of my COSC 330 final project from start to finish. I wrote this so that anyone — even someone brand new to Linux — can follow along and reproduce everything I built. Every command you need to type is in a code block. I explain what each one does so it's not just copy-pasting blindly.

The project is broken into 4 parts:

- **Part 1** – A C shell program that uses a custom static and shared library
- **Part 2** – A systemd background service running on Linux
- **Part 3** – A TCP file transfer between two terminals (simulating two machines)
- **Part 4** – Uploading, listing, and downloading files from a local S3-compatible storage server (MinIO)

Before Anything – One-Time Setup

First, open a terminal. On most Linux systems you can do this by pressing `Ctrl + Alt + T` or searching "Terminal" in your apps.

Make sure you have the tools installed. Run this to install everything at once:

```
sudo pacman -S --needed gcc make python-pip wget
```

When it asks for your password, type it and press Enter. The cursor won't move while you type — that's normal, it's a security thing.

Then install the Python library needed for Part 4:

```
pip install boto3 botocore
```

`boto3` is the library that lets Python talk to S3-compatible storage servers. I used it to upload and download files in Part 4.

How to Navigate to the Project

Before running anything, make sure you're in the right folder. Run this once when you first open a terminal:

```
cd ~/Documents/Final_for_SP
```

`cd` stands for "change directory." The `~` symbol is a shortcut that means your home folder (like `/home/yourname`). If you ever want to check what folder you're currently in, type:

```
pwd
```

That stands for "print working directory" and it just tells you where you are.

Part 1 – Building and Running the Shell with Libraries

This part shows how I built a reusable C library with math and string operations, compiled it into both a static (`.a`) and shared (`.so`) library, and then wrote a menu-driven shell that loads the shared library at runtime.

Step 1 – Go into the part1 folder

```
cd ~/Documents/Final_for_SP/part1
```

Step 2 – Compile everything with make

```
make
```

You should see output like this:

```
gcc -Wall -Wextra -fPIC -c mylib.c -o mylib.o
ar rcs libmylib.a mylib.o
gcc -shared -o libmylib.so mylib.o
gcc -Wall -Wextra -fPIC -o shell shell.c -L. -lmylib -Wl,-rpath,.
```

What just happened: `make` read the Makefile and ran the right `gcc` commands automatically. It compiled `mylib.c` into an object file, then packed that into a static library (`libmylib.a`) using `ar`, then built a shared library (`libmylib.so`), and finally compiled the shell and linked it to the shared library.

If you see any errors, the most common fix is making sure `gcc` is installed (`sudo pacman -S gcc`).

Step 3 – Prove the shell is using the shared library

```
ldd ./shell
```

Look for this line in the output:

```
libmylib.so => ./libmylib.so
```

`ldd` stands for "list dynamic dependencies." It shows which shared libraries a program needs at runtime. This line proves the shell is actually loading `libmylib.so` when it runs — which is the whole point of a shared library.

Step 4 – Run the shell

```
./shell
```

The `./` at the front just means "run this file from the current folder." You should see the menu pop up:

```
===== My Math & Utility Shell =====
1. Add two numbers
2. Subtract two numbers
3. Divide two numbers
4. Reverse a string
5. Fibonacci number
6. Exit
Choose an option:
```

Here is what to try to show all the features working:

- Type `1`, press Enter, then enter `10` and `5` — you should get `Result: 15`
- Type `2`, press Enter, then enter `10` and `3` — you should get `Result: 7`
- Type `3`, press Enter, then enter `10` and `4` — you should get `Result: 2.5`
- Type `3` again, but this time enter `10` and `0` — you should get `Error: division by zero` (this shows error handling works)
- Type `4`, press Enter, then type `hello` — you should get `Reversed: olleh`
- Type `5`, press Enter, then type `10` — you should get `fibonacci(10) = 55`
- Type `6` to exit

All of these operations (`add`, `subtract`, `divide`, `reverse_string`, `fibonacci`) are defined in `mylib.c` — not in `shell.c`. That's what makes it a proper reusable library.

Part 2 – Setting Up a systemd Service

This part shows how Linux manages background services using systemd. systemd is the first process that starts when Linux boots up (it runs as PID 1) and it's responsible for starting and stopping every service on the system — things like SSH, networking, and now my custom shell service.

Every command in this section needs `sudo` because managing system services requires administrator access.

Step 1 – Make the daemon script executable

```
chmod +x ~/Documents/Final_for_SP/part2/myshell_daemon.sh
```

`chmod +x` gives the file permission to be executed as a program. Without this, Linux won't let you run it.

Step 2 – Copy the service file into the systemd folder

```
sudo cp ~/Documents/Final_for_SP/part2/myshell.service
/etc/systemd/system/myshell.service
```

`/etc/systemd/system/` is the directory where systemd looks for custom user-defined services. We need `sudo` here because only administrators can write to `/etc/`.

Step 3 – Tell systemd to pick up the new service file

```
sudo systemctl daemon-reload
```

Any time you add or change a `.service` file, you have to run this. It tells systemd to re-read all the unit files so it knows about the new one.

Step 4 – Start the service

```
sudo systemctl start myshell
```

Step 5 – Check that it's actually running

```
sudo systemctl status myshell
```

You should see something like this:

```
● myshell.service - My COSC330 Shell Service
     Loaded: loaded (/etc/systemd/system/myshell.service; disabled)
     Active: active (running) since Tue 2026-05-12 14:32:01 CDT
   Main PID: 12345 (myshell_daemon.s)
```

The important part is `Active: active (running)` highlighted in green. That means the service is up and running.

Step 6 – Watch the live logs with journalctl

```
sudo journalctl -u myshell -f
```

`journalctl` is systemd's log viewer. `-u myshell` filters it to only show logs from my service. `-f` means "follow" — it streams new log lines as they come in, kind of like a live feed. You should see a new line appear every 30 seconds that looks like:

```
[2026-05-12 14:32:31] myshell service is running | host=nick-pc | uptime=up
2 hours
```

Press `Ctrl + C` when you're done watching.

Step 7 – Enable the service to auto-start on boot

```
sudo systemctl enable myshell
```

This tells systemd to automatically start `myshell` every time the computer boots up.

Step 8 – Stop and clean up

```
sudo systemctl stop myshell
sudo systemctl disable myshell
```

`stop` kills the process now. `disable` removes it from the boot sequence so it won't auto-start next time.

Part 3 – TCP File Transfer Between Two Terminals

This part demonstrates socket programming. I wrote a server and a client in C that communicate over a real TCP connection. On actual EC2 instances these would be two separate cloud computers — here I'm running them on the same machine using two terminal windows to simulate that.

You need **two terminal windows open side by side** for this part.

Step 1 – Build the server and client

In either terminal, run:

```
cd ~/Documents/Final_for_SP/part3
make
```

This compiles both `server.c` and `client.c` into executable programs.

Step 2 – Create a test file to send

```
echo "Hello from the client! Sending file1.txt over TCP on port 9000." >
file1.txt
```

`echo` just prints text, and the `>` redirects that output into a file instead of the screen. This is the file we're going to transfer.

Step 3 – Start the server in Terminal 1

In **Terminal 1**:

```
cd ~/Documents/Final_for_SP/part3
./server
```

You'll see:

```
Server listening on port 9000...
```

Leave this sitting here. The server is now waiting for a connection. Don't close this terminal.

Step 4 – Connect and send the file from Terminal 2

Switch to **Terminal 2** and run:

```
cd ~/Documents/Final_for_SP/part3
./client 127.0.0.1 file1.txt
```

`127.0.0.1` is called the loopback address — it means "connect to this same computer." On real EC2 instances you would replace this with the server's private IP address (something like `10.0.1.25`).

Terminal 2 (client) should print:

```
Connected to 127.0.0.1:9000 — sending 'file1.txt' (67 bytes)
Transfer complete: 67 bytes sent
```

Terminal 1 (server) should print:

```

```

```
Connection from 127.0.0.1
Expecting 67 bytes
File saved as 'received_file1.txt' (67 bytes)
```

Step 5 – Confirm the file transferred correctly

```
cat received_file1.txt
```

`cat` just prints a file's contents to the terminal. It should show the exact same text you put in `file1.txt`. That confirms the file made it through the TCP connection intact.

How it works under the hood: the client first sends an 8-byte header with the file size (so the server knows how many bytes to expect), then streams the file in 64KB chunks. This is called data framing — without it, TCP has no way to know where one message ends and another begins.

Part 4 – File Upload and Download with Local S3 Storage (MinIO)

This part shows how to interact with object storage, which is how cloud applications store files at scale. Instead of using AWS S3 (which requires an account and costs money), I'm using **MinIO** — a free, self-hosted program that acts exactly like S3. The Python code and the underlying API are identical to what you'd use with real AWS.

You need a **third terminal** for MinIO since it has to stay running the whole time.

Step 1 – Download the MinIO binary

```
wget https://dl.min.io/server/minio/release/linux-amd64/minio -O /tmp/minio
chmod +x /tmp/minio
```

`wget` downloads a file from the internet and saves it to `/tmp/minio`. Then `chmod +x` makes it executable.

Step 2 – Create a folder for MinIO to store its data

```
mkdir -p ~/minio-data
```

`mkdir -p` creates a directory. The `-p` flag means "create any parent folders too if they don't exist yet."

Step 3 – Start the MinIO server in Terminal 3

Open a third terminal and run:

```

```

```
MINIO_ROOT_USER=minioadmin MINIO_ROOT_PASSWORD=minioadmin /tmp/minio server
~/minio-data --console-address ":9001"
```

MinIO will start and show some startup info. It listens on port 9000 for API requests and port 9001 for the web interface. Leave this terminal running the entire time — closing it would shut down the storage server.

You can open a browser and go to `http://localhost:9001` to see the MinIO web dashboard. Log in with `minioadmin` / `minioadmin`. This is a great thing to show in the demo because it looks like the real AWS S3 console.

Step 4 – Run the transfer script

Go back to your main terminal and run:

```
cd ~/Documents/Final_for_SP/part4
python3 s3_transfer.py
```

You should see all 4 steps complete:

```
[1] Created 'output.txt':
Timestamp : 2026-05-12T19:45:00Z
Hostname  : nick-pc
System    : Linux 6.x.x
Machine   : x86_64

[*] Bucket 'cosc330-bucket' created.
[2] Uploaded 'output.txt' -> s3://cosc330-bucket/output.txt

[3] Contents of s3://cosc330-bucket:
    output.txt  (108 bytes,  2026-05-12 19:45:01+00:00)

[4] Downloaded s3://cosc330-bucket/output.txt -> 'downloaded_output.txt'

Done.
```

Step 5 – Verify the downloaded file

```
cat downloaded_output.txt
```

The contents should match `output.txt` exactly. This proves the full round trip worked: create → upload → list → download.

Step 6 – Check the MinIO web interface

Go to `http://localhost:9001` in a browser, log in, click on `cosc330-bucket`, and you'll see `output.txt` listed there with its file size and upload timestamp. This is the equivalent of what the AWS S3 console would look like on a real EC2 instance.

Troubleshooting

These are the most common issues I ran into while testing everything:

Problem	What to do
`make: command not found`	Run `sudo pacman -S make gcc` to install the compiler
`./shell: error while loading shared libraries: libmylib.so`	You have to run `./shell` from inside the `part1/` folder so it can find `libmylib.so`
`Permission denied` on systemctl commands	Add `sudo` in front — service management requires admin access
`Connection refused` when running the client	The server has to be running first in Terminal 1 before you run the client
MinIO download fails with `wget`	Check your internet connection, or download the MinIO binary manually in a browser
`ModuleNotFoundError: No module named 'boto3'`	Run `pip install boto3 botocore`
Part 4 fails with a connection error	MinIO has to be running in Terminal 3 before you run `s3_transfer.py`
`sudo: systemctl: command not found`	Your system doesn't use systemd — this shouldn't happen on standard Linux distros

---

Quick Reference – All Commands Back to Back

If you've already done the setup before and just need the commands fast:

```
# ── PART 1 ──────────────────────────────────────────────
cd ~/Documents/Final_for_SP/part1
make
ldd ./shell
./shell

# ── PART 2 ──────────────────────────────────────────────
chmod +x ~/Documents/Final_for_SP/part2/myshell_daemon.sh
sudo cp ~/Documents/Final_for_SP/part2/myshell.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl start myshell
sudo systemctl status myshell
sudo journalctl -u myshell -f
```

```
# (Ctrl+C to stop following logs)
sudo systemctl stop myshell
sudo systemctl disable myshell

# ── PART 3 ── (Terminal 1 = server, Terminal 2 = client) ─
cd ~/Documents/Final_for_SP/part3
make
echo "Hello from the client! Sending file1.txt over TCP." > file1.txt
# Terminal 1:
./server
# Terminal 2:
./client 127.0.0.1 file1.txt
cat received_file1.txt

# ── PART 4 ── (Terminal 3 = MinIO, stays running) ─────────
# Terminal 3:
MINIO_ROOT_USER=minioadmin MINIO_ROOT_PASSWORD=minioadmin /tmp/minio server
~/minio-data --console-address ":9001"
# Main terminal:
cd ~/Documents/Final_for_SP/part4
python3 s3_transfer.py
cat downloaded_output.txt
```

Nicholas Armenta — COSC 330 Systems Programming — Spring 2026